01 What Is a World Model?

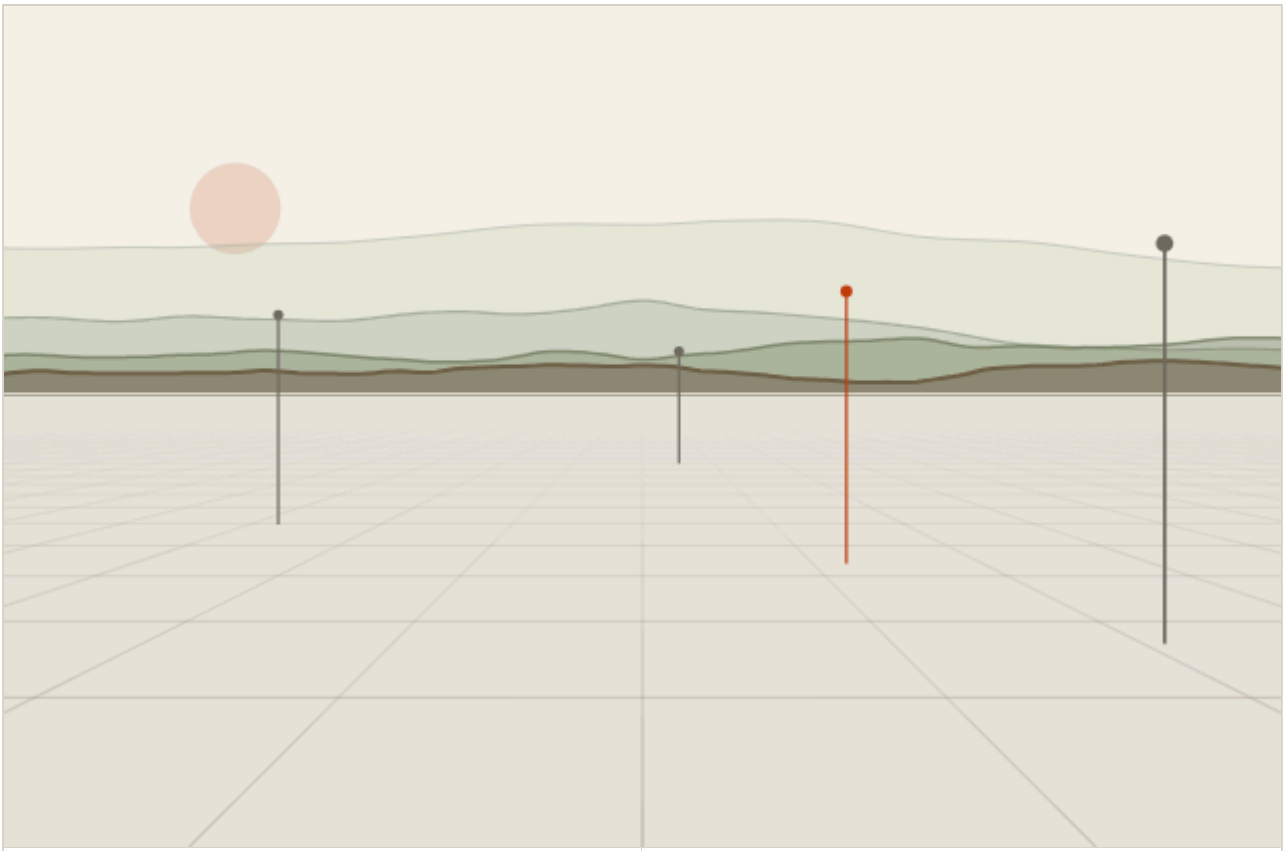
BY NILUSHANAN KULASINGHAM

24 MIN READ · INTERACTIVE

> The phrase has covered at least five different machines since 2018, and the people using it rarely say which. A field guide to the five, where each came from, and the one test that separates them.

The figures in this chapter are interactive. Press and drag them online at worldmodels101.com/chapters/what-people-mean.

If you've spent any time on your timeline lately, you'll probably have seen a clip like the one below. Somebody is walking through a landscape with the arrow keys, and the caption says it was generated rather than built. It looks like a video game that nobody made. The replies call it a world model, so you go and look the phrase up.



HEADING

0°

FROM START

0.0 strides

FIG. 1.1 Nothing in this picture is a recording. Every frame is drawn from where you stand and which way you face, and nothing behind it stores a landscape. Drag to look around, or click it and walk with the arrow keys.

Ten minutes later you have five tabs open, and they describe five different machines. One generates video that you steer with the arrow keys. One exports geometry, meaning 3-D shapes and where they sit, into Blender, the 3-D modelling program. One is a paper about predicting embeddings, lists of numbers that stand in for pictures, and it never shows you a video at all.

I should say the phrase confused me for a long time too. The explanations online, while helpful, tended to pick one of the five machines and explain it as if it were the whole story. What I wanted was one page that laid all five out side by side and said how they came to share a name. This chapter is that page, and if you are confused like I was, hopefully it helps.

The good news is that the mess has a history, and the history is a lot tidier than the tabs. Between about 2018 and 2024, four traditions arrived at the same two words from four directions. They were reinforcement learning and control, self-supervised representation learning (teaching networks to summarise images and video), generative video and spatial models, and interpretability, the study of what goes on inside a trained model. Each meant something real, and none of them had to check what the others meant.

“world model” 5 INCOMPATIBLE ANSWERS

A DeepMind demo where somebody walks through a generated landscape with the arrow keys.

Genie 3 · Google DeepMind ↗

TURNS OUT TO BE

The Renderer
https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

A Meta paper about predicting video embeddings that never shows you a video.

V-JEPA 2 · Meta AI ↗

TURNS OUT TO BE

The Representation
https://ai.meta.com/blog/v-jepa-2-world-model-benchmarks/

A 3-D environment you can load straight into Blender.

Marble · World Labs ↗

TURNS OUT TO BE

The Simulator
https://www.worldlabs.ai/blog/marble-world-model

A reinforcement learning result from 2018 about a car in a racing game.

World Models · David Ha & Jürgen Schmidhuber ↗

TURNS OUT TO BE

The Dynamics Model
https://worldmodels.github.io/

An argument, conducted mostly at volume, about whether a language model that has never seen a chessboard has one inside it anyway.

Emergent World Representations · Kenneth Li et al. ↗

TURNS OUT TO BE

```
The Implicit Model
https://arxiv.org/abs/2210.13382
```

FIG. 1.2 Have a look at these five results, from five different classes of system. Every one of them was called a world model by the people who built it.

By 2026 the labs had started publishing dictionaries. Five definitions are in current use, each out of its own tradition and its own year. There is a four-second test that ought to tell them apart, and as we'll see, it does not.

There are two things to say before we start. Firstly, I'm going to assume that you have seen a few of these clips and used a chatbot or two, and not much more than that. Anything more technical I will explain as we go, and there is less of it than you might expect.

Secondly, I'll be leaning on the figures. Each one is something you can press and drag, and I'll tell you what to press and what you should expect to see. The chapter makes a lot more sense if you do.

One equation, sixty years

Let's start with where the term came from, because it started somewhere specific. Everything after it either grew from that start or pushed against it. The start was control theory, the engineering maths of keeping a machine on course: an autopilot, a thermostat, a rocket.

Later it moved into reinforcement learning (training an agent by letting it act, watching what happens, and rewarding what worked). There, a world model is a learned transition function. All that means is a rule for what comes next, one the agent picks up from experience.

A state and an action go in, and the next state comes out. When I say state, I mean a description of how things stand at one moment. Written down, it looks like this.

$$p(s_{t+1} \mid s_t, a_t)$$

the chance of what happens next, given where you are now and what you do

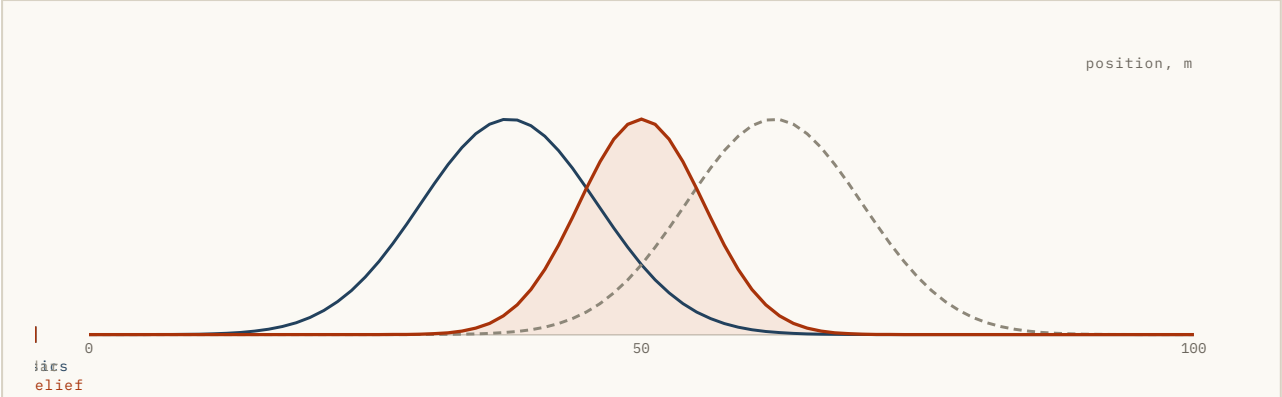
FIG. 1.3 Every system below is really an argument about what belongs in place of *s*. It could be pixels, 3-D geometry, a short list of numbers, or an embedding. That choice is the main axis the five definitions separate along, so keep it in mind.

The oldest ancestor is the Kalman filter, from 1960. Rudolf Kalman, a Hungarian-born engineer, set it out in *A New Approach to Linear Filtering and Prediction Problems*. A filter here is a piece of maths that cleans up noisy readings. If you feed it a radar's blips on a plane, it keeps a running best guess of where the plane is and how fast it is going.

Within a decade it was flying in Apollo's guidance computer. It is still the maths inside a GPS receiver. I'd say that is one half of what a world model does. There is a hidden state you never see directly, and the job is to estimate it from what you can see.

The trick inside it is the same one that runs in every GPS receiver, drone and self-driving car, so let's walk through it. At each moment you have two stories about where the plane is. One comes from physics: take the last position and the last speed, and predict where the plane should be now. The other comes from the radar, which hands you a blip.

Both stories are uncertain. Each is really a bell curve, a Gaussian, with the most likely spot in the middle and the less likely ones fading out either side. Multiply the two curves together and you get a third bell curve, which sits between the two and is narrower than either. That is the new belief.

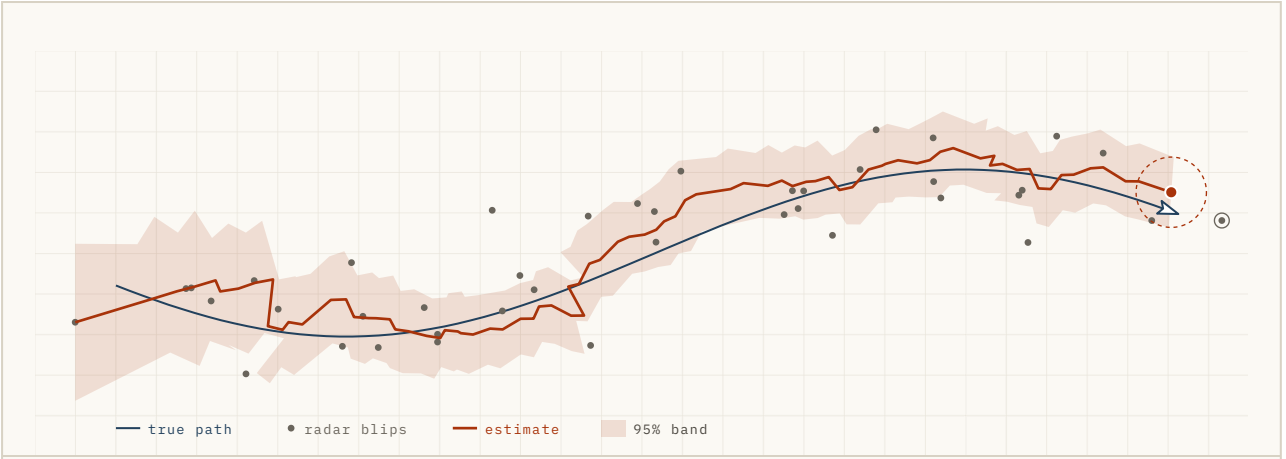


KALMAN GAIN K	NEW MEAN	NEW WIDTH
0.50	50.0 m	σ 5.7 m

FIG. 1.4 Here are the two stories about where the plane is, drawn as bell curves on one axis. Slide how sure physics is and how noisy the radar is, and watch the new belief lean toward the sharper story while staying narrower than either. The gain is the number that says how far it leaned. Now press Correct to make the new belief the next physics prediction, and watch it widen again as the plane moves on.

How far the new belief leans from physics toward the radar is set by one number between zero and one, which the field calls the Kalman gain. As you can see from the figure, a noisy radar pulls the gain toward zero, so the filter sticks with physics. A sharp radar pulls it toward one, so the filter trusts the reading.

Then it repeats: predict, correct, predict, correct. Every new blip, however wrong on its own, tightens what the filter knows. The estimate hugs the truth even though every reading it ever got was off, and it does that one tick at a time without ever waiting for the future.



The band is tighter than the radar's scatter: the filter trusts its own dynamics as much as the radar, and it carries a speed it never measured.

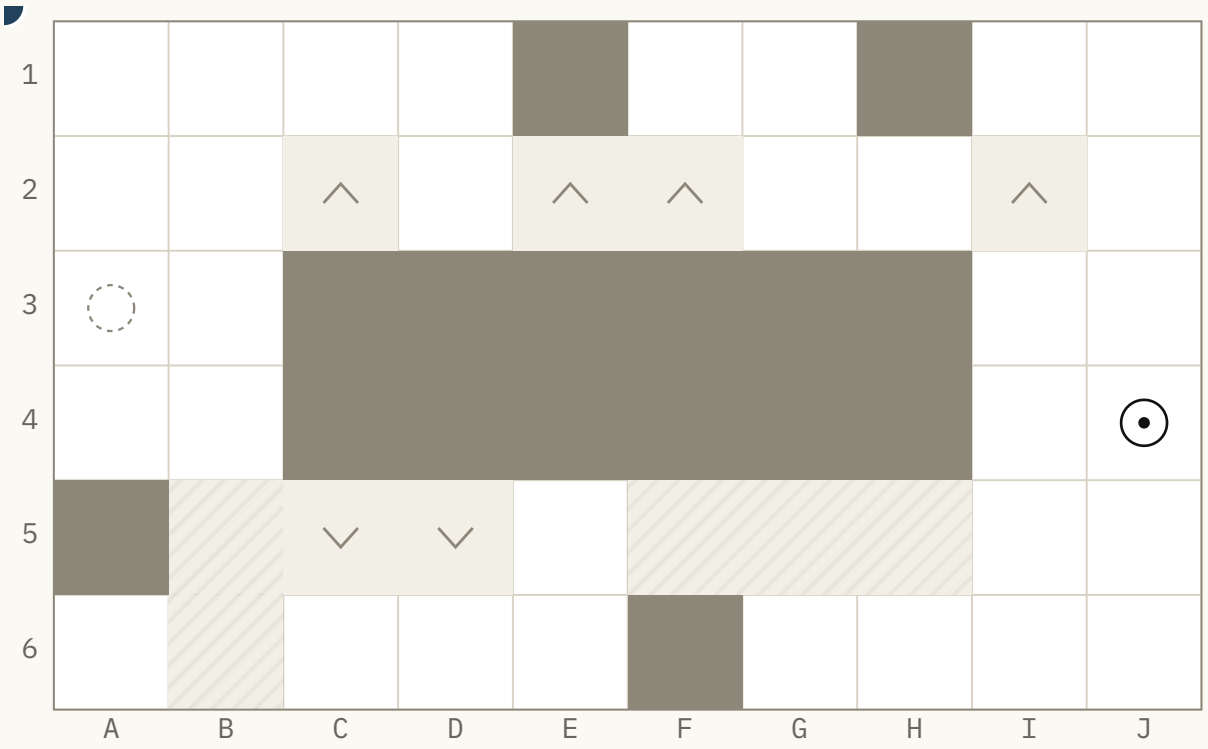
LATEST BLIP OFF BY	ESTIMATE OFF BY	BAND, 95%
420 m	140 m	±260 m
SPEED, ESTIMATE / TRUE	GAIN, PHYSICS 0 TO RADAR 1	
110 / 105 m/s	0.27	

FIG. 1.5 A plane flies along a smooth path, and a radar reports its position every couple of seconds with noise on top. A Kalman filter keeps a running guess of where the plane is and how fast it is going. Scrub the time or press play, and watch the band around the estimate tighten as the blips arrive. Turn the radar noise up, and the estimate still sits closer to the plane than the blips do. Drop the blips for a stretch and it coasts on its own dynamics while the band opens. The gain readout is the lean: near zero it trusts physics, near one it trusts the radar. The greyed-out control is the point I want you to notice: the filter can propagate a turn you hand it, but nothing here learns the dynamics or weighs one turn against another. Distances are illustrative.

What it does not do is the other half. You can hand a Kalman filter a known control input, a turn you have decided on, and it will propagate it, so it can tell you where that turn would take the plane. But the rules for how the state moves are written in by the engineer rather than learned, and nothing in it compares two turns to pick one. It is an estimator, not a planner, and its dynamics are supplied rather than learned. That is why I call it an ancestor rather than a member.

The other half took thirty years and a different field. By 1990 neural networks, programs that learn from examples rather than from rules someone typed in, were back in fashion. Three groups reached for the same design in the same year, so let's take them one at a time.

Jürgen Schmidhuber, then a young researcher in Munich, built a system in two parts. His paper was *Recurrent world models for planning and curiosity*. One network learned to predict what the world would do next. A second one chose actions, and the first told it what each choice would lead to.



WORLD MODEL

s, a ⟶ s′

One square per move. Knows the walls, not the patches.

CHOOSER

Plans the route inside the model. Takes its first step.

No laps finished yet. Reaching the goal ends a lap and the dot goes back to the start.

LAP	STEPS THIS LAP	SURPRISES THIS LAP	CORRECTIONS HELD
1	0	0	0

FIG. 1.6 This is the 1990 design in miniature. Point at an arrow and the world model draws where it thinks you would land. Press it, and the world answers. The model knows about the walls but not the patches: ice carries you on, and drift and lift push you off your row. Now switch on "Learn from mistakes" and press "Run a lap" three times. You should see the surprises fall lap by lap, because

> the chooser now plans inside a model that knows the route. Then press "Forget", switch learning off, and run three more: every lap is as surprised as the first.

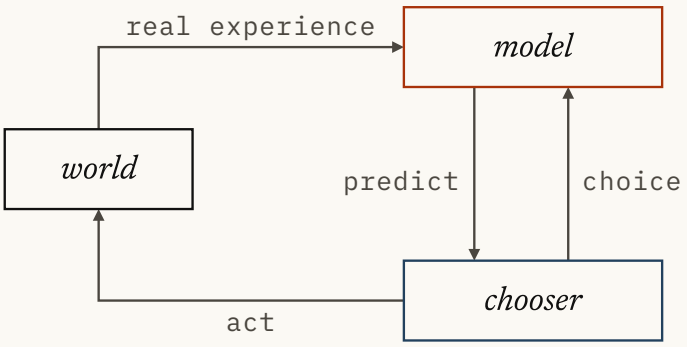
He called the first network the world model, and that is the phrase's birth certificate. Schmidhuber, who later co-invented the LSTM memory cell, has spent a career insisting the field forgets who had its ideas first. On this one, the record is plainly his.

In the same period Richard Sutton built Dyna. It was a learning agent that practised partly on real experience and partly on experience its own model made up. His paper was *Dyna: an Integrated Architecture for Learning, Planning and Reacting*.

Sutton and Andrew Barto went on to write the textbook every reinforcement learning student still reads. The two shared the 2024 Turing Award, announced the following spring.

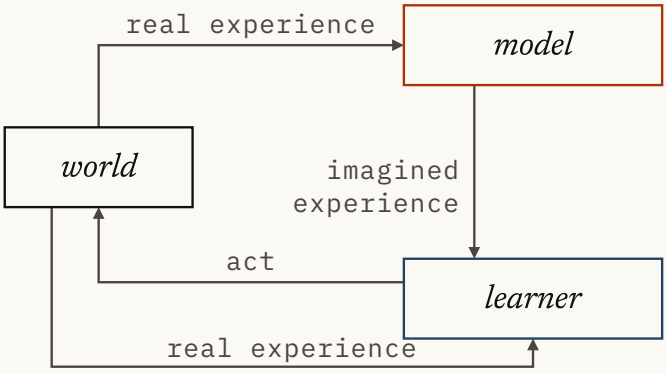
JÜRGEN SCHMIDHUBER 1990

Recurrent world models for planning and curiosity



RICHARD SUTTON 1991

Dyna, an Integrated Architecture for Learning, Planning and Reacting



SEBASTIAN THRUN, KNUT MÖLLER AND ALEXANDER LINDEN 1990

Planning with an Adaptive World Model

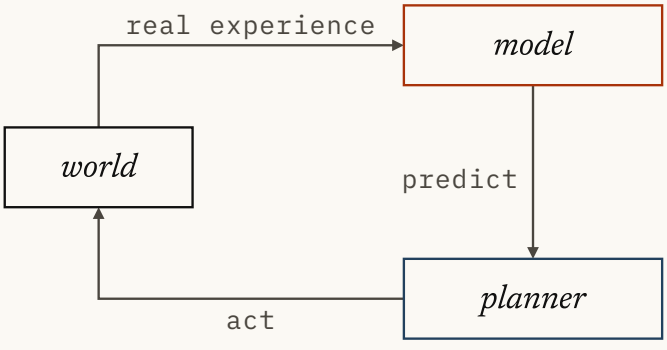
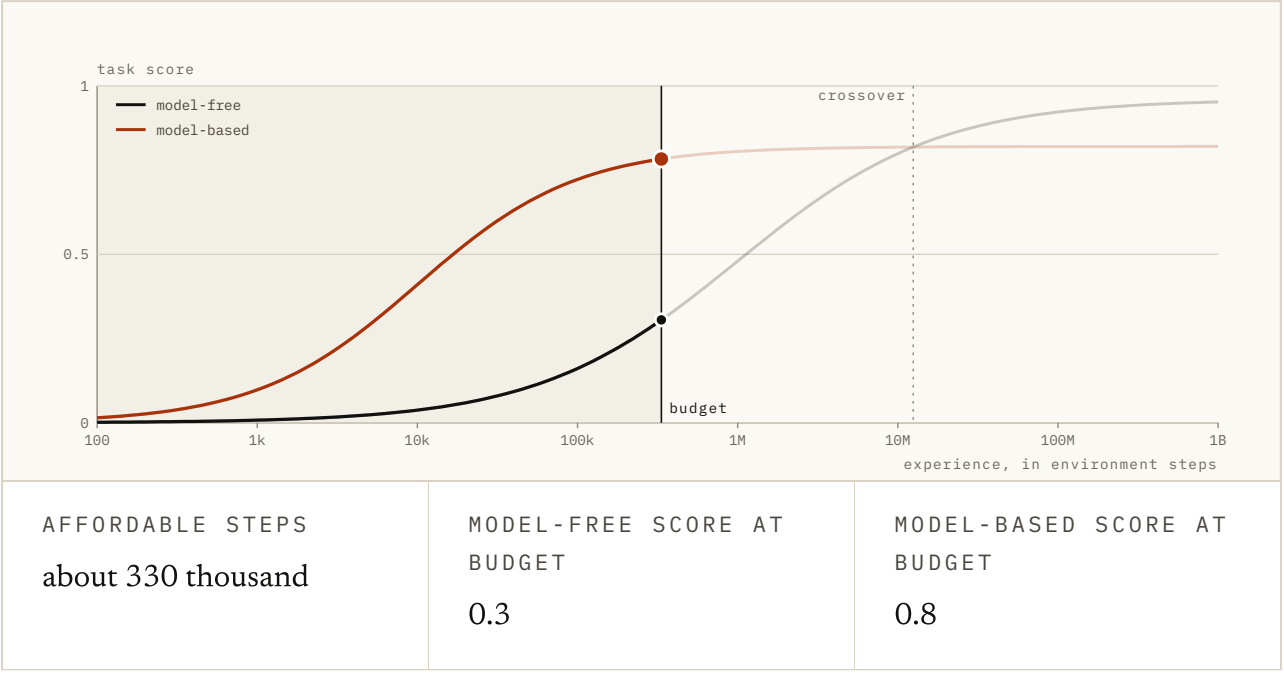


FIG. 1.7 Here are the three systems from 1990 and 1991, each drawn as a loop. Click a card to enlarge it and read what its model does and what uses it. Then turn on the shared shape and watch: the world and the arrows fade, and the same two boxes light up on every card.

Sebastian Thrun, Knut Möller and Alexander Linden made the third, in *Planning with an Adaptive World Model*. As you can see from the figure, all three had the shape of the modern idea. A learned model says what happens next, and something else uses it to decide.

The label did not stick, though. The networks of 1990 were small, and their worlds were mazes and a few dozen numbers. Nobody could learn a useful model of anything with a camera in it.

For most of the next twenty-five years the model-based line, meaning agents that carry a model and use it, stayed a minority interest. When deep learning reached reinforcement learning, the headline result was DeepMind's DQN in 2013. It learned Atari games from pixels with no model of the game at all. Model-free had the momentum, and the two-part design of 1990 looked like a relic.



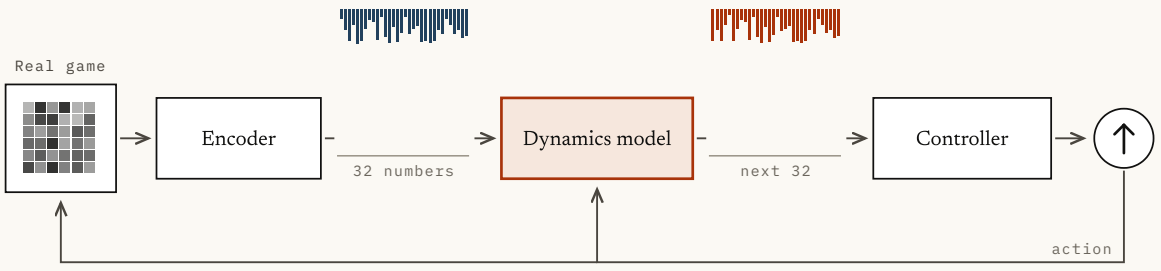
AFFORDABLE STEPS	MODEL-FREE SCORE AT BUDGET	MODEL-BASED SCORE AT BUDGET
about 330 thousand	0.3	0.8

FIG. 1.8 The curves here are shapes rather than measurements. Pick what one step costs, then drag the budget and watch where the shaded region stops. In a simulator, a program that imitates the world, the budget reaches past the crossover and the slow climber wins. On a robot it stops long before that, and the only agent that has learned anything is the one carrying a model. Then switch on the bad model to see what happens when the model is wrong in the places you go.

David Ha and Jürgen Schmidhuber's *World Models* (2018) is the paper that made the label popular. I read it as a reply to that momentum. Ha was then at Google Brain. The paper showed the 1990 design working on a car-racing game and a level of the video game Doom, in three pieces.

The first is an encoder (a network that squeezes a video frame down to a short list of numbers), thirty-two numbers here. The second is a dynamics model, which predicted where those numbers go next. The third is a small controller, the piece that picks actions.

The controller was trained almost entirely inside the model's own imagined rollouts, futures the model played out for itself. Then it was moved back into the real game, and it still worked.



PIECE | ENCODER | DYNAMICS MODEL | CONTROLLER

IN The thirty-two numbers, plus the action taken.

OUT The next thirty-two numbers.

FOR Predicts where those numbers go next. It is the one piece that can run on its own output.

REAL STEPS	IMAGINED STEPS	LATENT WIDTH
0	0	32
		Ha and Schmidhuber's choice

FIG. 1.9 Ha and Schmidhuber's three pieces, drawn as a loop. Select a box to see what goes in and what comes out, then press Step and watch the thirty-two numbers change. Now flip the switch to run inside the dream: the dynamics model feeds its own prediction back, and the game is no longer consulted. That is how the controller was trained before it was moved back into the real game. Then have a go at the question underneath.

One thing that can mislead you in the papers is the naming. Ha and Schmidhuber call their third module the controller, meaning the small policy that picks actions. The world model is the module beside it, the one the controller plans inside. If you called the whole thing "the controller" you would name it after the one piece that is plainly not a world model.

PlaNet came the same year, from Danijar Hafner and colleagues, with Ha among the authors. The paper was *Learning Latent Dynamics for Planning from Pixels*. It learned those dynamics straight from pixels. Then it planned in latent space (latent just means the model's own compressed description: a few hundred numbers instead of a million pixels).

At every step it tried many action sequences inside the model and took the best first move. Dreamer (2019) came from the same group, in a paper called *Dream to Control*. Instead of searching for a plan at every step, it learned its behaviour from futures imagined inside the model.

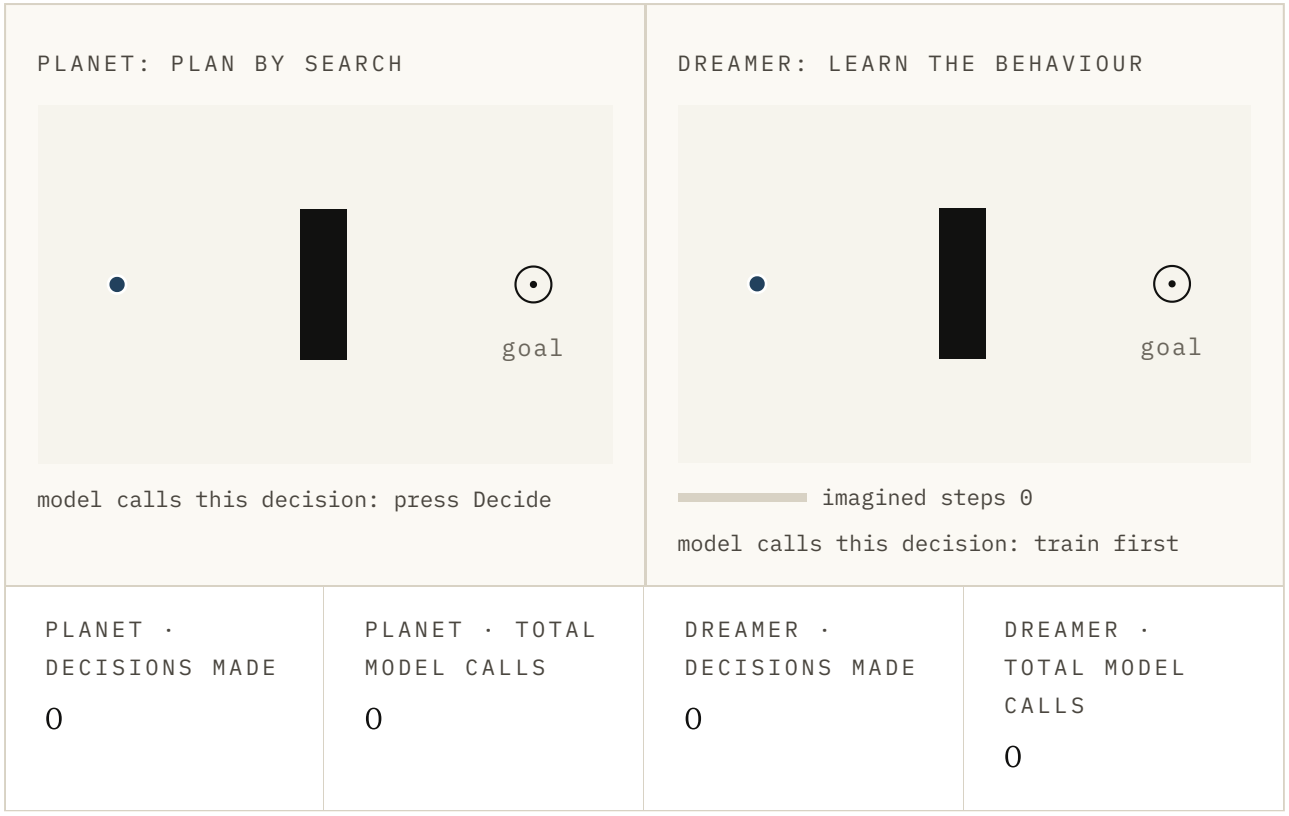
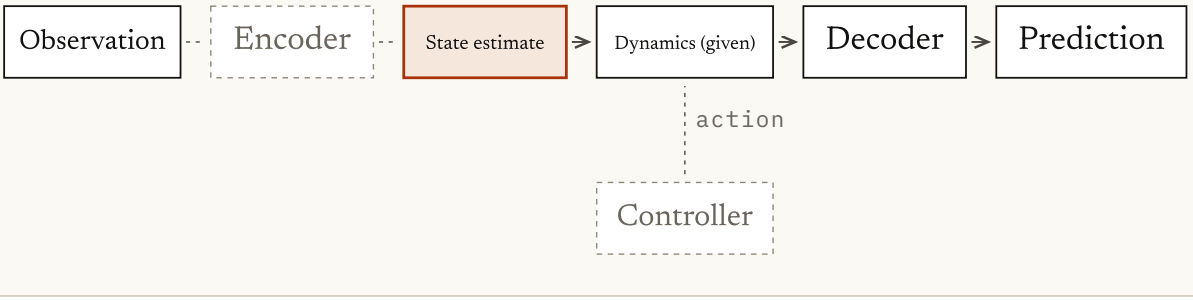


FIG. 1.10 Two copies of the same small world. On the left, press Decide: the model imagines twelve futures, keeps the best, and the dot takes one step, so every decision costs seventy-two model calls. On the right, press Train in imagination once and then Decide: the behaviour was learned in advance, so each step costs the model nothing. The thing to watch is the two total-calls counters rather than the dots.

Its third version reached *Nature* in 2025 with one set of settings across more than 150 tasks. That is as close to a dynasty as this field gets. By the end of 2019, though, the reinforcement learning reading of the word was settled.

Then the word travelled, in two directions at once. In 2022 Yann LeCun published *A Path Towards Autonomous Machine Intelligence*. He was Meta's chief AI scientist at the time, and he is a Turing Award winner. He argued that predicting every pixel of the next frame is the wrong job, because most pixels are detail nobody can predict.

JEPA, the family of models that followed at Meta, predicts a summary of the next frame instead. That lets it drop the decoder, which is the part that would have turned the prediction back into pixels.



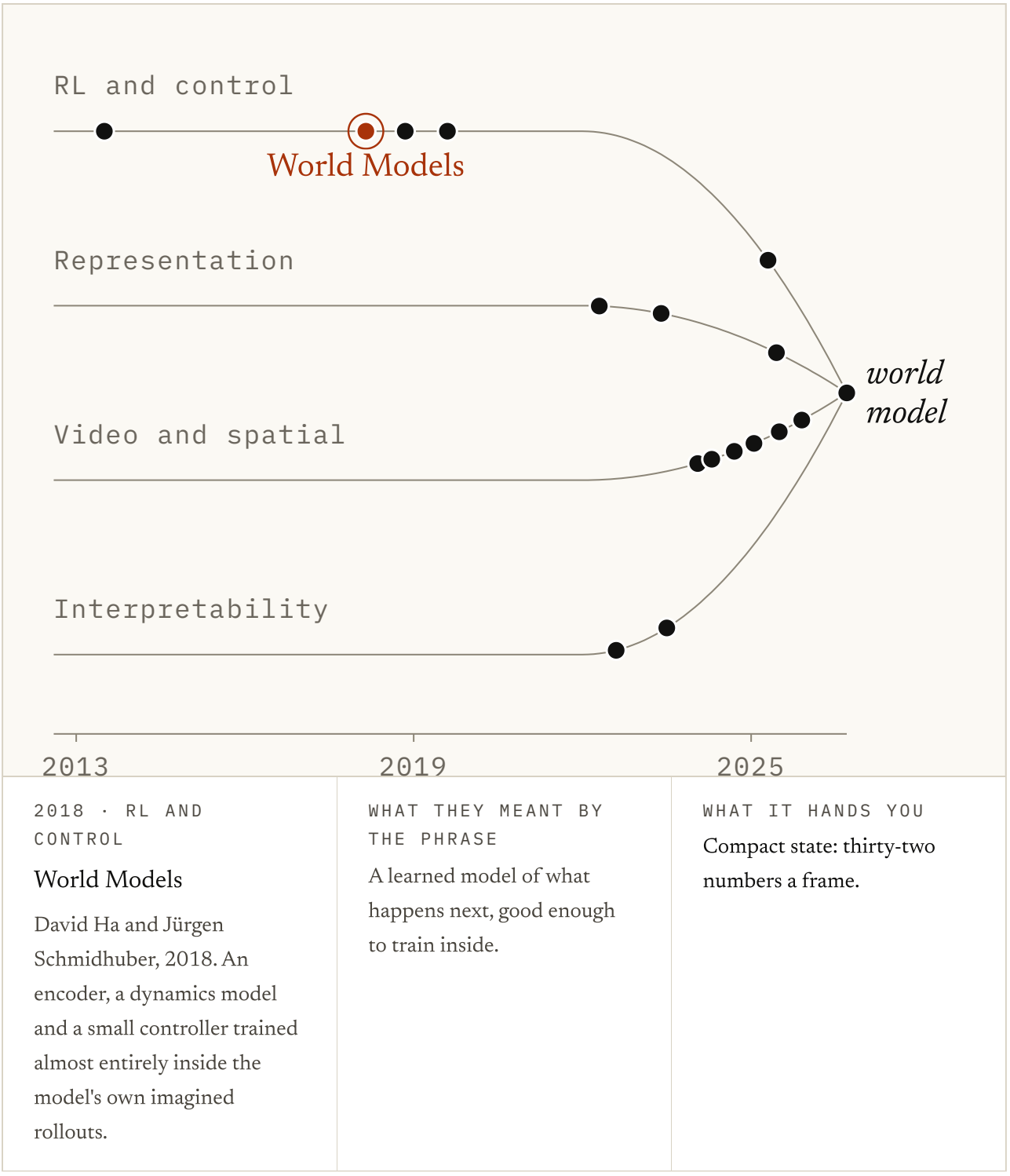
Recover a hidden state from noisy measurements by running a forward model and correcting it against what you observe. The dynamics are supplied, not learned, and nothing here learns them or chooses between actions.

FIG. 1.11 Scrub through the years and watch the parts. Each era switches a part on, relabels it or points it at a new target, and now and then one switches a part off. Nothing ever gets replaced, and that is how systems built for different reasons ended up sharing a name.

The other direction went toward worlds you can see and walk through. In February 2024 OpenAI published Sora with a report titled *Video generation models as world simulators*. The same month Genie arrived from Jake Bruce and colleagues at DeepMind. It learned its own set of actions from unlabelled internet video of games, then let you play what it drew.

Cosmos from NVIDIA and Marble from World Labs followed in 2025. Meanwhile the interpretability people had borrowed the word for something with no demo at all. Their claim was that a model trained for one job had grown a model of the world inside itself. We'll come back to that one.

By June 2026 World Labs had published *A Functional Taxonomy of World Models*, a sorting system for the mess. The dictionaries had begun. Four traditions each held a different piece of one problem. From the outside, the term looks confused.



2018 · RL AND CONTROL

World Models

David Ha and Jürgen Schmidhuber, 2018. An encoder, a dynamics model and a small controller trained almost entirely inside the model's own imagined rollouts.

WHAT THEY MEANT BY THE PHRASE

A learned model of what happens next, good enough to train inside.

WHAT IT HANDS YOU

Compact state: thirty-two numbers a frame.

FIG. 1.12 Drag the year back to 2013 and come forward. For most of the decade only one lane has anything on it, and the guides only begin to bend after 2022. Click a dot to see who used the phrase, what they meant by it, and what their system hands you. As you can see, the lines meet at one phrase rather than at one thing.

The four only noticed they were neighbours when their outputs started to look alike. By then each of them had been using the word for years.

The five definitions

Now let's go through the five. Treat these categories as contracts rather than species. Each one describes what a system promises to hand you. Modern work crosses between them all the time, as NVIDIA's Cosmos and Meta's V-JEPA 2 are about to show.

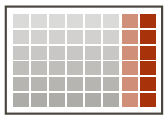
FIG. 1.13 Four of these are systems you can run. They are ordered by how abstract the predicted object is. The fifth is a claim about another system rather than a system itself, which is why it sits off the axis.

The Renderer predicts observations, usually pixels. GameNGen, from Dani Valevski and colleagues at Google (2024), draws frames of the video game DOOM. Each frame comes from the frames before it and the player's inputs. It runs at about twenty frames a second on a single TPU chip, and that is fast enough to play.

Genie 3, from DeepMind, is reported to generate interactive video at 720p and 24 frames per second. It is said to stay coherent for several minutes, with the scene hanging together as you move. It also takes plain-language instructions mid-session: change the weather, add a flock of birds.

Sora belongs in this category too, but without the action input: it predicts observations you cannot steer. Keep that difference in mind, because whether actions change the prediction is one of the three questions at the end of this chapter.

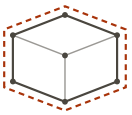
So persistence exists, meaning things stay where you left them. It is learned through generation, though, rather than guaranteed by stored geometry.



PREDICTS PIXELS

The Renderer

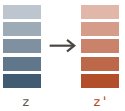
A frame, then the next frame. What comes out is something to look at, and the room persists only because it kept drawing it consistently.



PREDICTS GEOMETRY & PHYSICS

The Simulator

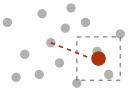
Geometry another program can open. Surfaces to render, and collider meshes underneath for a physics engine to bump into.



PREDICTS COMPACT STATE

The Dynamics Model

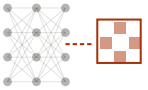
A short vector, and the next one under an action you are considering. Nobody names the numbers. It only has to be rollable forward.



PREDICTS EMBEDDINGS

The Representation

The summary of the missing piece, predicted as a point in embedding space and then thrown away. It never redraws the pixels.



OFF THE AXIS

The Implicit Model

Not an output at all. A probe finds board state inside a network nobody handed a board to, which is a claim about what arose, not a thing you can run.

FIG. 1.14 The same five, this time drawn by what each one hands you. Vermilion, the orange-red, marks the predicted object in every panel. Left to right is the same axis as the map above: a picture, then structure, then a compact state, then a summary, and finally something that is not an output at all.

The Simulator predicts structure you can query, meaning 3-D shapes you can ask questions of. Marble comes from World Labs, a start-up co-founded by the Stanford researcher behind ImageNet. That is the picture dataset that set off the deep learning boom. Marble takes text, an image or a rough 3-D layout, and hands back two things at once.

Gaussian splats (millions of coloured blobs, cheap to render, nothing to bump into) carry the look. Triangle meshes carry the structure. A mesh is just a surface built from many small triangles. That is ordinary 3-D geometry, collider meshes included (the invisible shapes a physics engine bumps into).

World Labs shipped Marble to a limited beta in November 2025 and opened it to everyone in February 2026. NVIDIA open-sourced Cosmos in January 2025. It generates physically-aware video, built as training data for robots and self-driving cars. It is also the case that keeps the map honest.

RENDERER / PREDICTS OBSERVATIONS

GOOGLE DEEPMIND Genie 3: Creating dynamic worlds that you can navigate in real-time

SIMULATOR / PREDICTS STRUCTURE

WORLD LABS Introducing Marble by World Labs

FIG. 1.15 I'd read this as a comparison of promises rather than a pass/fail test. Marble hands you geometry that another program can open, so you can check it yourself. Genie 3's coherence is reported by the lab that built it, and you cannot check that. The difference is what each one promises, and how much of it you can check.

NVIDIA itself calls Cosmos Predict, the part that makes the video, generative video. The real 3-D simulator is Omniverse, and that is a different NVIDIA product. If you put the whole thing under "simulator" you have hidden that split from yourself.

The lesson is that a platform is not a category. One product can expose a pixel predictor, a 3-D simulator and an action-conditioned model all at once. Action-conditioned just means it takes your actions as input. Ask which one you are being sold, and which interface you are about to build against.

The Dynamics Model predicts compact state under actions, often with the reward. Compact means a short list of numbers rather than a picture. It is the oldest of the four runnable categories. It is the one the line from Schmidhuber to Dreamer was building toward.

MuZero, from Julian Schrittwieser and colleagues at DeepMind, is the proof. Its model predicted only reward, value and likely moves, never the board. With that it matched AlphaZero, its famous predecessor, at Go, chess and shogi. It was never told the rules.

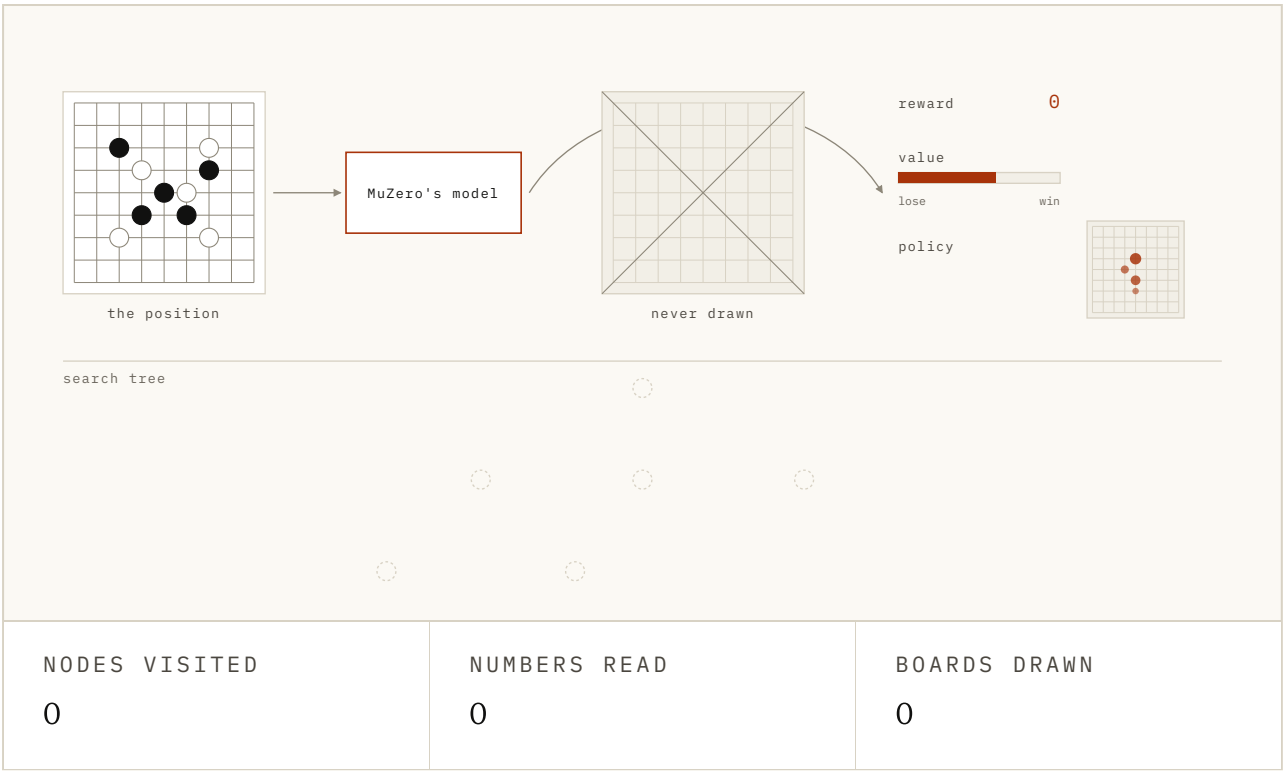
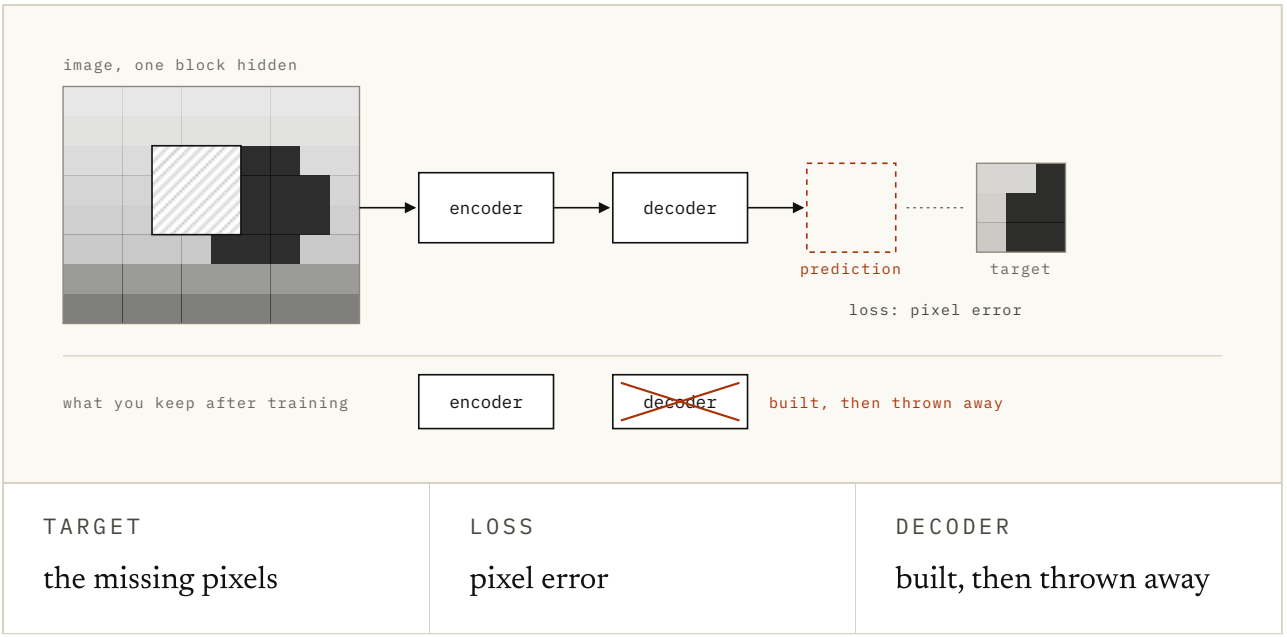


FIG. 1.16 MuZero's model sits in the middle. Leave the switch on three numbers and press Search to step a small tree: every node asks for reward, value and policy, and the board on the right is never drawn. Now flip the switch to the next board and watch each node pay for a picture that the search then reads nothing from. The values are illustrative.

PlaNet plans inside a model like this, searching afresh at every step. Dreamer learns behaviour from futures imagined through one. What matters here is whether you can search inside it. How real it looks does not come into it.

The Representation predicts embeddings (a list of numbers standing in for an image or a clip, with similar things close together). Then it throws the prediction away. I-JEPA (2023), the image version from Mahmoud Assran and colleagues, hides part of an image and predicts the embedding of the missing piece. It never redraws the pixels.

V-JEPA 2 (2025), the video version, pre-trains on more than a million hours of video with no actions involved. A second stage then trains an action-conditioned variant on top. That one is built for model-predictive control (plan a few steps, take one, plan again). Meta reports it driving a Franka robot arm in a lab it never saw during training.



TARGET	LOSS	DECODER
the missing pixels	pixel error	built, then thrown away

FIG. 1.17 Choose what the network is asked to predict for the hidden block, then press Predict. With pixels as the target, a decoder has to turn the prediction back into a picture, and the best it can do for a block it cannot see is a blur. With a summary as the target, nothing is turned back into pixels. Look at the bottom row, which is what survives training: the encoder in both cases, and only one of them ever built a decoder.

Meta also reports it planning around 30× faster than Cosmos. Be careful with that number: it compares two different contracts rather than ranking them. Anyone quoting it as a ranking has not read the contracts.

Notice that the second stage takes actions, rolls forward and gets searched over. That is the Dynamics Model's contract, reached from the other side. I call it migration: a system starts in the category it was trained for and grows the machinery of another on top. The categories describe a training target rather than a fixed identity.

The Implicit Model is a different kind of claim altogether. It is the least discussed of the five, because it has no demo.

Othello-GPT is a small language model trained only to predict legal moves in Othello, a board game played with two-sided discs. It was never shown a board. Kenneth Li and colleagues (2022), who trained it, reported in *Emergent World Representations* that the board was there inside it anyway.

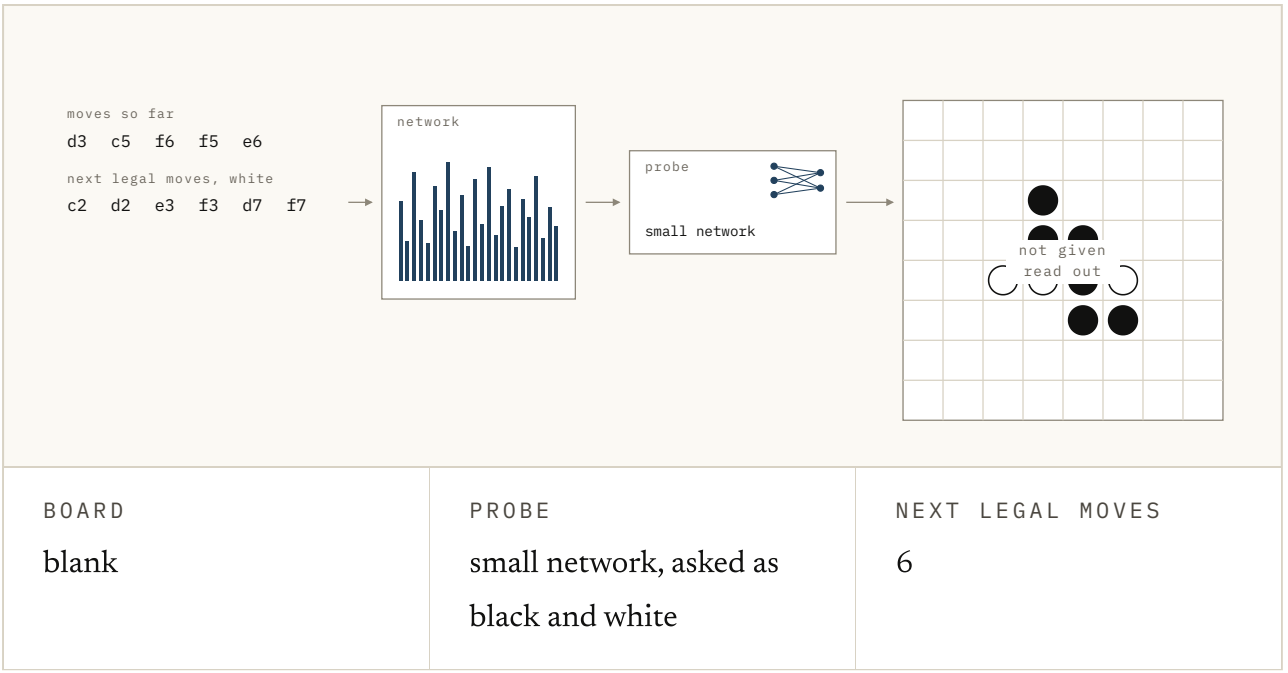


FIG. 1.18 Othello-GPT only ever saw move tokens like the strip on the left. Press Read the board and a probe draws an 8 by 8 board out of the network's activations, and nobody put one in. Flip a square in that inner board and watch the list of next legal moves change, which is how Li and colleagues showed the model was playing from it. Then switch to mine and theirs, which is Nanda's reframing, and the probe can be a straight line.

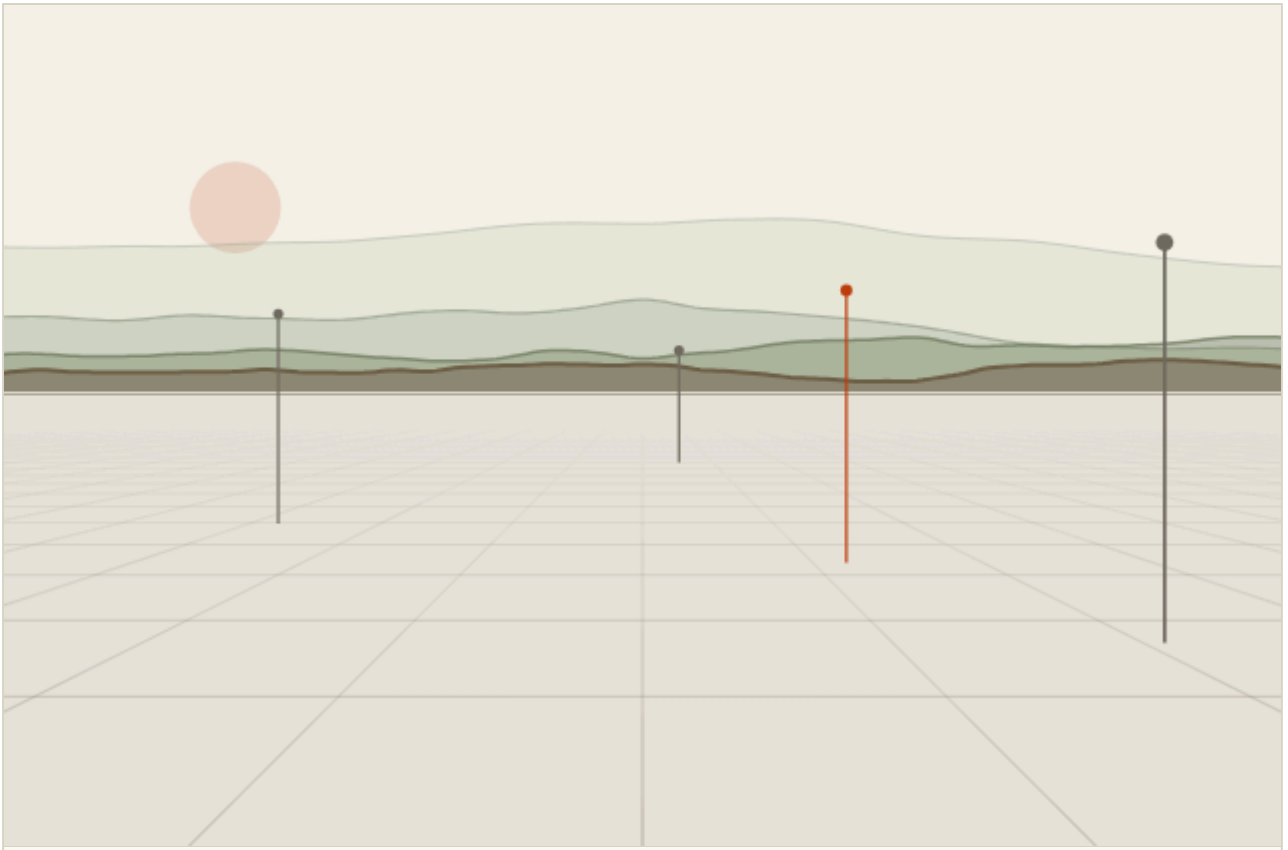
A probe (a small classifier trained to read one specific fact out of a network's activations) could read it out. When they changed that inner board by hand, the model's next moves changed too.

Neel Nanda and colleagues (2023) followed up, in a paper called *Othello-GPT has a linear emergent world representation*. The board was there in a form plain enough for the simplest probe to read. You only had to ask for "mine" and "theirs" instead of black and white.

Nobody handed the model a function called `world_model`. The phrase here is a claim about what grew inside another system, so it is a finding rather than an interface.

The test that was supposed to settle it

One test ought to sort all five out, and it takes about four seconds. Turn around and walk away, then turn back: is it the same room? Something that holds a world inside it keeps the furniture where you left it, and something that is only drawing pictures cannot. I call it the turn-around test.



WHAT IS STORED

nothing. each frame is drawn from the heading

HEADING	FROM START	LANDMARK
0°	0.0 strides	13° right, 5.5 strides

FIG. 1.19 The turn-around test, made playable. With the switch off, press Turn away and back and look at where the vermilion (orange-red) post comes back. Then switch the world on and run it again. The panel on the right shows you what is stored in each case.

It does not work, and the figure above shows why. Leave the switch off and press "Turn away and back". You should see that the post has drifted, because nothing behind the picture is holding it there.

Now switch "Hold the world" on and do it again. The post stays put, because a stored coordinate is being read back. Excellent.

So those are the two ways to pass the test. You can store the room and look it up when the viewer turns back. Or you can get very good at drawing a room that matches the one you drew a moment ago. From the outside they look the same, and only the switch tells them apart.

A large model trained on a lot of video learns the second way. Nobody gave it a room to keep. It got good at continuing what it had already started, and continuing consistently is part of that. That is why Genie 3's persistence is real, and why the turn-around test was never going to catch it.

> DeepMind reports this of Genie 3: scenes holding together over several minutes, and details coming back when you return to them. I'd take it as their report rather than something anyone outside the lab can check.

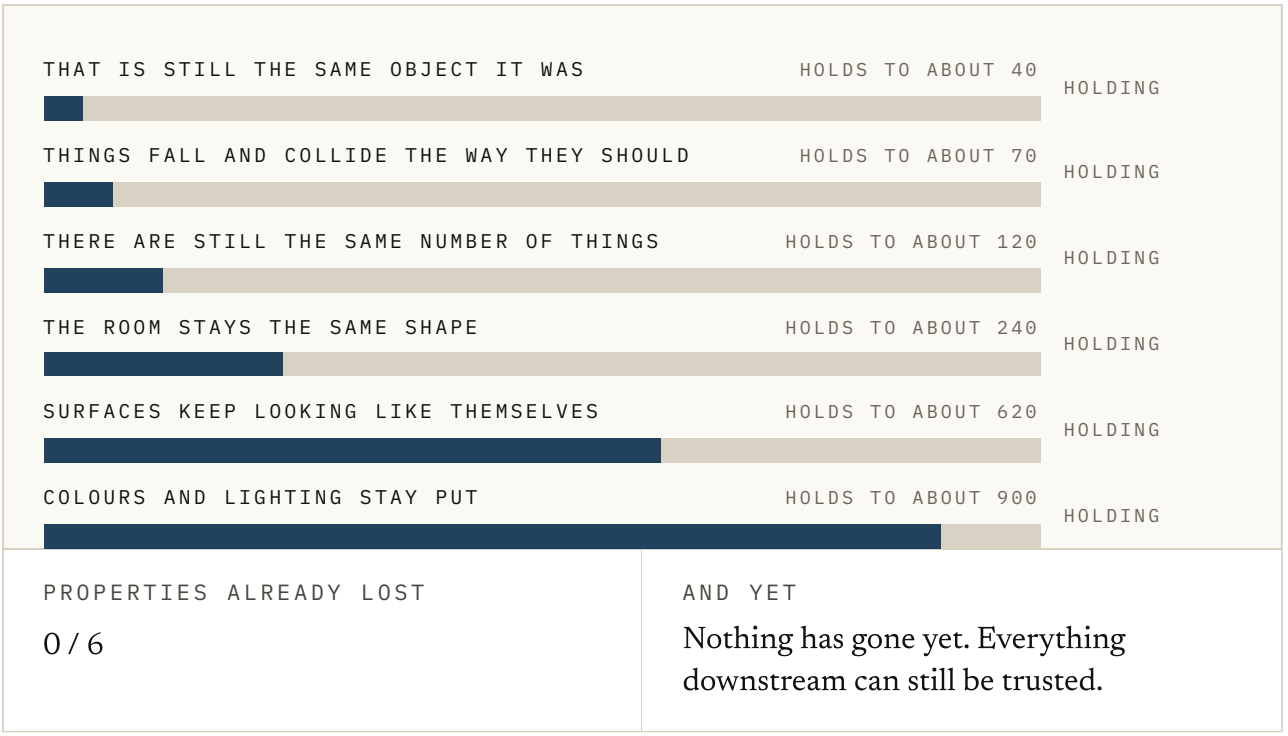


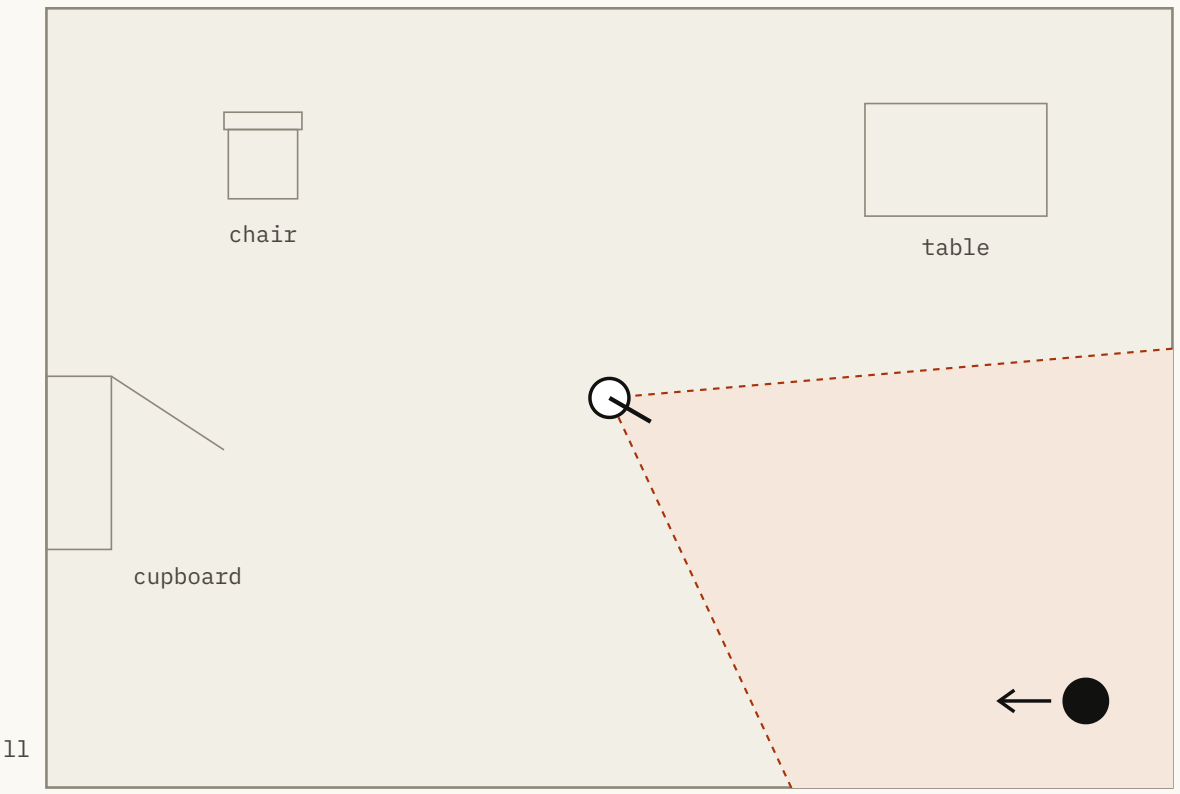
FIG. 1.20 This is persistence learned through generation, watched over a long run. Drag out to a thousand steps and read off which properties are still holding: texture and count go last, and identity and physics go first. Nothing here was stored; all of it was continued.

So the turn-around test cannot sort them, but it was reaching for the right distinction. When you turn away from the chair, the chair does not stop existing. It only stops being *visible*. Those are two different things, and nearly everything hard about this subject lives in the gap between them.

What is there, whether or not you can see it, is called the **state**. The part of it that you can see right now is called the **observation**.

> State is the underlying reality of the world; complete in principle, but never directly visible to any agent inside it. Observations are an agent's partial view of that reality.
>
> World Labs on the distinction underneath the taxonomy (A Functional Taxonomy of World Models, June 2026)
> https://www.worldlabs.ai/blog/taxonomy-of-world-models

You never get the state, only observations, and you work backwards from them. Think of the chair behind you, a ball still rolling out of shot, or whether the cupboard is open. All of it is real and none of it is visible.



	STATE what is there	OBSERVATION what the camera sees now
CHAIR	here	?
BALL	rolling left, at 7.4 m	rolling left, at 7.4 m
CUPBOARD	open	?
TABLE	here	?

HEADING	IN VIEW	TIME STEP
30°	1 of 4	0

FIG. 1.21 Turn the camera with the slider and watch the two columns. The state column never changes what it lists. The observation column only shows what falls inside the wedge, and marks the rest with a question mark. Now turn away from the ball, press Step time once or twice, then turn back: the ball has moved, and your last observation of it was wrong the whole time.

Working out the rest from the part you can see is called **partial observability**.

None of this is new, and the date is the part I like best. A Cambridge psychologist, Kenneth Craik, described the fix in 1943, in a book called *The Nature of Explanation*.

He was twenty-nine when it came out. Two years later he was dead, knocked off his bicycle on a Cambridge street. That was seventeen years before Kalman, and long before there was anything to build it out of.

> If the organism carries a 'small-scale model' of external reality and of its own possible actions within its head, it is able to try out various alternatives, conclude which is the best of them, react to future situations before they arise... and in every way to react in a much fuller, safer, and more competent manner to the emergencies which face it.
>
> Kenneth Craik on internal models, forty years before anyone trained one (The Nature of Explanation, 1943)

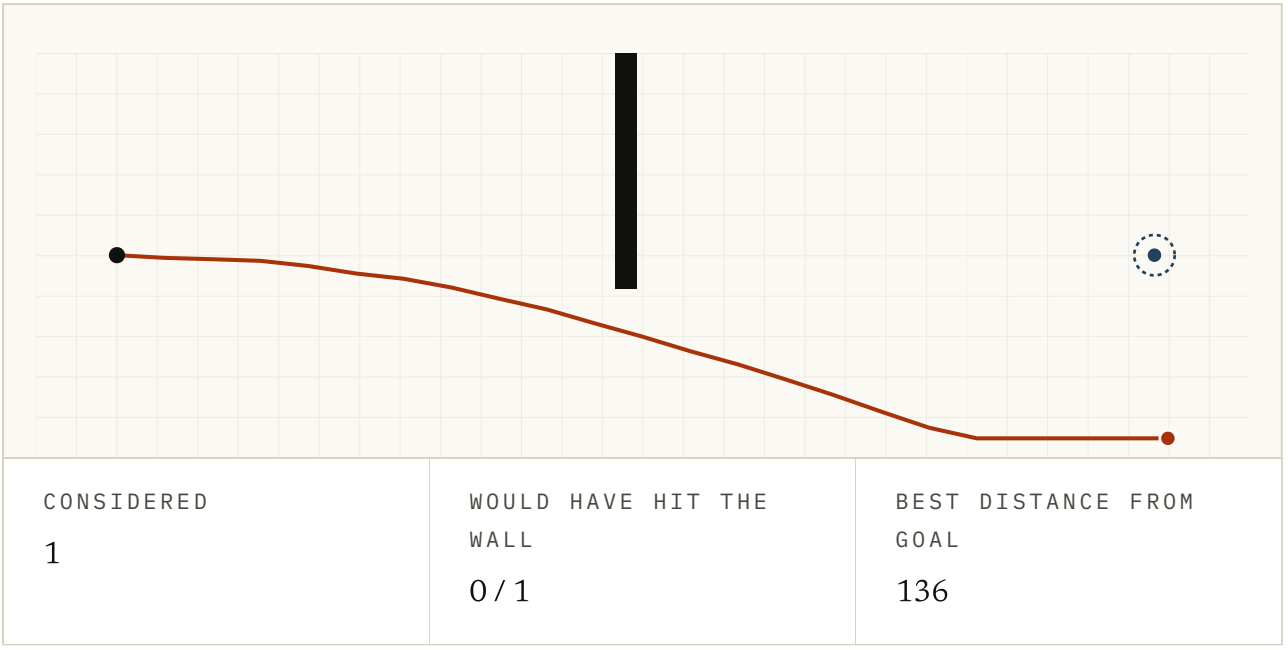


FIG. 1.22 Craik's small-scale model, run by a machine. The planner writes down runs of actions, plays each one through the model, and keeps the best. Raise the budget and you should see the rejects fade and the plan sharpen.

If you carry a small model of the world in your head, you can test an action before you take it. Every definition on the map is a different bet on what Craik's small-scale model should hold.

What they were all fighting over

As we determined above, one object sits underneath all five definitions. World Labs derive their own taxonomy from the same object, and Sutton's Dyna was running round it back in 1990.

An environment has a state. An agent receives observations that reveal part of it, and from observations and memory it builds an internal state. A model predicts how things could change. The agent acts, the world changes, and a new observation arrives.

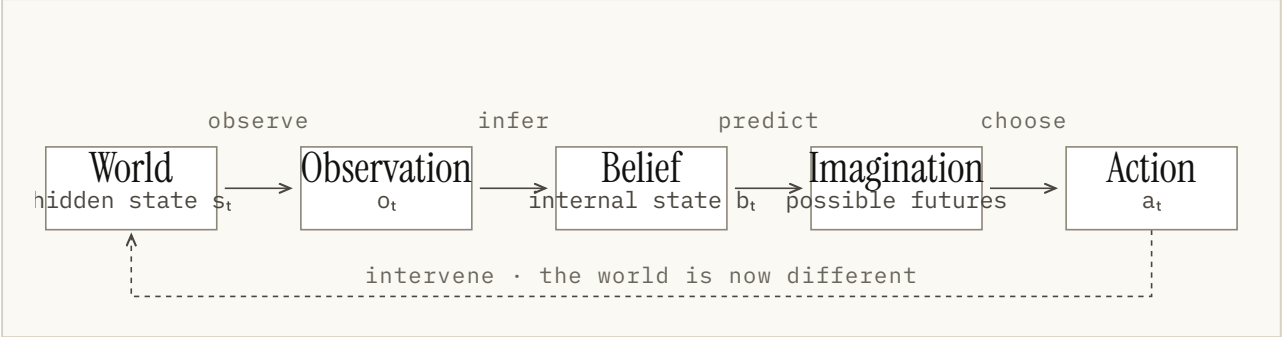


FIG. 1.23 The agent-environment loop, with each definition shown as a specialist on one arc. Systems that look incompatible share a name because each one owns a piece of the same loop.

If you draw the loop that way, three problems fall out of it. They are the same three problems the field has been arguing about since Kalman wrote down a filter in 1960. Each gets its own chapter later, so for now I'll give you the short versions.

How sure should it be? Throw a ball behind a wall and ask where it is now. A single exact position is the wrong shape of answer, because you cannot know. It might have bounced off something, or stopped against a kerb, or still be rolling.

A model that hands back one confident coordinate has thrown away the fact that it was guessing. PlaNet carries two things at once instead. One part is deterministic (computed the same way every time, so it always follows from the last state). It holds whatever reliably follows from what came before.

The other part is stochastic (sampled rather than computed, so the same input can produce several different futures). It holds whatever could still go either way. You can think of the split as the model admitting what it does not know.

TIME HIDDEN	WIDTH OF THE SPREAD	THE SINGLE DOT
0 steps	collapsed to a point	on the ball

FIG. 1.24 Drag time forward and watch the ball go behind the wall. The faint dot is the answer a deterministic model gives: one coordinate, moved on at the known speed. Switch on the stochastic part and the model also carries a spread, which widens the longer the ball is hidden. Keep going past the point where the dot comes out, and ask yourself which answer was honest.

Do actions change the prediction? *What happens next* and *what happens if I push this* are two different questions. Only the second one lets you compare options before choosing.

A model that answers it is called action-conditioned (it is told which action was taken, so it can answer what-if instead of only what-next). That is what control needs. As we saw earlier, Kalman's filter could propagate a control input you gave it, but it never learned the dynamics or weighed one action against another. The learning and the choosing came later.

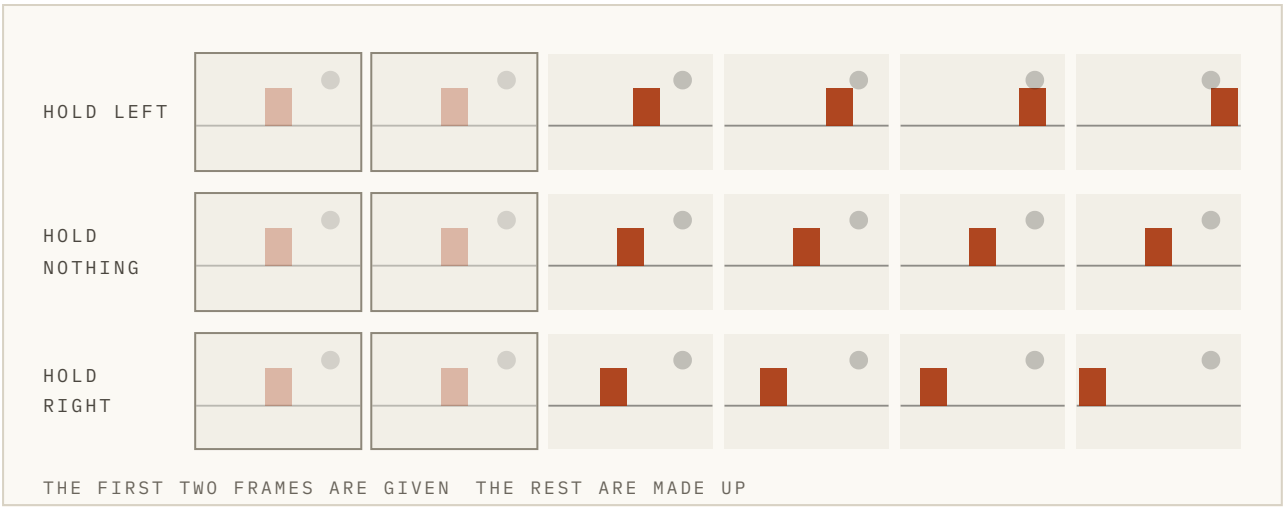


FIG. 1.25 One start and three futures, chosen by which key you hold. Turn the action input off and watch the three rows fold into one: without an action there is only what probably happens next, and no longer what happens if you do this.

How far can it roll forward? One prediction is rarely enough. To plan, a model has to feed its own answer back in and predict again, and then again, on top of what it just made up. The number of steps you ask for is called the **horizon**.

Errors pile up along that stretch. Nothing dramatic happens at any single step, which is what makes the problem hard to notice. The model is only ever slightly wrong. Get step one slightly wrong, though, and step twenty can be somewhere else entirely.

$$p(s_{t+1} \mid s_t, a_t) \implies p(s_{t+1:t+H} \mid s_t, a_{t:t+H-1})$$

Ask for the next state instead of every state from here to H, and you still only get the same single state you are in and a whole plan.

FIG. 1.26 The same model, this time asked for a stretch of future instead of a single step. Two things grow and one does not, and that is the whole difficulty.

Dreamer exists to learn behaviour across many imagined steps. DeepMind names long-horizon coherence as the big open problem for its generated worlds. That means a scene that stays true to itself over many steps.

Dreamer is a latent dynamics model from 2019, and Genie 3 is a pixel renderer from 2025. They sit at the two ends of the map, fighting the same problem from opposite sides.

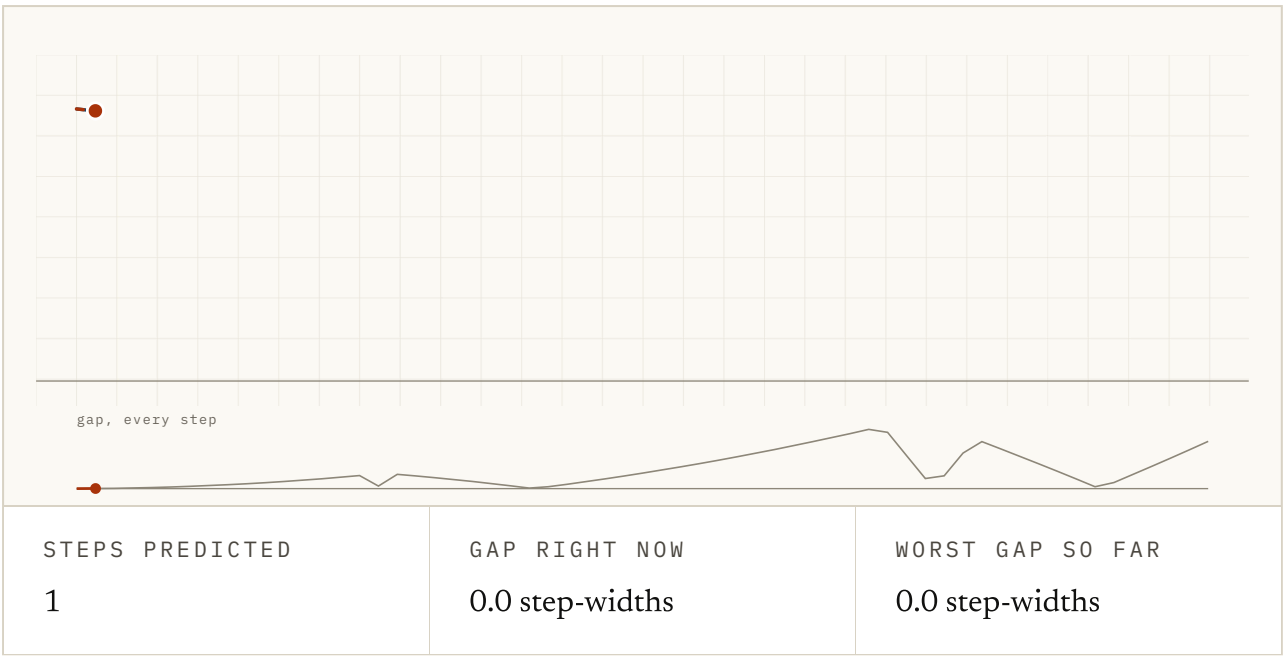


FIG. 1.27 Both dots start in the same place under the same rules, except that one is running dynamics that are a few percent off. Drag to about 8 and you'll see they are still together. Keep going, and notice that nothing breaks at any single step. The gap is what piles up.

So sixty years on, the open problem is drift, the very thing Kalman's filter was built to correct. The difference is that an imagined rollout has no fresh readings to correct against.

Now let's go back to the clip. It is a Renderer, and you can say why: it predicts observations. It holds the room because it learned to, and nothing in it promises geometry underneath.

The next one that goes past your timeline will be called a world model too. The question to ask is which of the five it is, and whether the person posting it could tell you.

Hopefully this chapter helped. There is a short quiz below if you want to check the ideas stuck. If I have got something wrong, or you know a better source than the ones I used, the About page has a corrections section. I would be glad to hear from you.

Try it

1 It takes one photo of a kitchen and returns a mesh you can import into a game engine, with collision volumes on the worktops.

`The Renderer · The Simulator · The Dynamics Model · The Representation · The Implicit Model`

> ANSWER The Simulator. Something else can open it and compute against it. Collision volumes are the giveaway: they exist for a physics engine to bump into, not for you to look at.

2 You are watching a camera feed of a room. There is a chair behind the camera. Where does that chair sit?

a. Outside the state, because nothing can see it
b. In the state but not the observation
c. In the observation but not the state
d. In neither, until the camera turns

> ANSWER b. In the state but not the observation. Turning away does not delete furniture. The chair is part of what is there, just not part of what you can currently see. The gap between those two is where most of the hard problems in this course come from.

3 You hold a key and it streams video of a city that has never existed, a frame at a time, reacting to which way you steer.

`The Renderer · The Simulator · The Dynamics Model · The Representation · The Implicit Model`

> ANSWER The Renderer. The output is the picture. It may well stay consistent as you drive, but nothing underneath is obliged to, and there is no city to hand anyone.

4 A system keeps the room consistent when you turn away and turn back. What has that proved?

a. It is storing the room
b. It is not storing the room
c. Nothing on its own
d. It must be a Simulator

> ANSWER c. Nothing on its own. There are two ways to pass that test, and from the outside they are identical. You can keep the room, or you can be very good at redrawing it. The result is the same, so the result cannot tell you which.

5 It is trained only to predict the next move in chess games. Researchers later probe it and find it tracks where the pieces are.

`The Renderer · The Simulator · The Dynamics Model · The Representation · The Implicit Model`

> ANSWER The Implicit Model. Nobody built a chess model here and nobody can run one. The claim is about structure found inside a network trained for something else, which is a claim of a different kind from all the others.

6 A model is slightly wrong at every step. You feed its own output back in twenty times. What happens to the error?

a. It stays about the same
b. It roughly doubles
c. It grows, unevenly, and can end up somewhere else entirely
d. It cancels out over enough steps

> ANSWER c. It grows, unevenly, and can end up somewhere else entirely. Each imagined state becomes the input to the next prediction, so mistakes are built on. Nothing dramatic happens at any single step, which is what makes it hard to catch.

7 It hides part of a video and learns to predict a summary of the hidden part. Once trained, the predictions are thrown away and the rest is bolted onto a robot.

`The Renderer · The Simulator · The Dynamics Model · The Representation · The Implicit Model`

> ANSWER The Representation. The forecast was scaffolding. What survives training is the way it learned to describe things, which is the product.

8 Going from predicting one step to predicting H steps, what does NOT get bigger?

a. The stretch of future you are asking for
b. The number of actions you have to supply
c. What you are given to start from
d. The number of ways it can go wrong

> ANSWER c. What you are given to start from. However far ahead you ask, you are still standing in exactly one place with one observation of it. The question grows; the evidence does not.

9 Given a compact state, a few numbers describing the scene, and a motor command you are considering, it returns the compact state you would be in next. A search loop calls it a few hundred times per decision.

`The Renderer · The Simulator · The Dynamics Model · The Representation · The Implicit Model`

> ANSWER The Dynamics Model. It hands you state rather than a picture, and you can roll it forward under actions nobody has taken yet. That is the Dynamics Model's contract, and the search loop is what it is for. Had it returned the next sensor reading instead, you would be looking at a Renderer.

10 Why does the Kalman filter count as ancestry rather than as one of the five?

a. It is too old to count
b. Its dynamics are supplied rather than learned, and it never compares actions to choose one
c. It cannot take a control input at all
d. It only works on linear systems

> ANSWER b. Its dynamics are supplied rather than learned, and it never compares actions to choose one. It does half the job well: work out a hidden state from noisy measurements, and it will even propagate a control input you hand it. What it never does is learn the dynamics or weigh one action against another, and those are what make the rest of these useful for choosing. Linearity is a detail of the original, not the reason.

11 One era in the history removed a part instead of adding one. Which, and why?

a. The encoder, because pixels stopped mattering
b. The decoder, because the prediction target moved off the pixels
c. The controller, because planning was abandoned
d. The dynamics, because they became implicit

> ANSWER b. The decoder, because the prediction target moved off the pixels. JEPA predicts a summary of the next frame rather than the frame, so nothing needs to turn the prediction back into pixels. That is the same reason the forecast can be discarded and the features kept.

12 A lab generates photorealistic video of motorway driving to train a self-driving stack. It is marketed for robotics.

`The Renderer · The Simulator · The Dynamics Model · The Representation · The Implicit Model`

> ANSWER The Renderer. This is the trap. Being aimed at robots suggests a Simulator, but the output is still video, with no geometry anyone can collide against. What a system is for is a weaker clue than what it hands you. Even that is not a complete answer, because a large platform can ship several interfaces, so the question is which one you are about to build against.

13 A lab reports its system stays coherent for several minutes. You cannot run it yourself. How should that sit in your notes?

a. As a fact, since they built it
b. As reported, not checked
c. As false until proven
d. As irrelevant to the category

> ANSWER b. As reported, not checked. This is not scepticism for its own sake. Some claims you can open and verify, like a mesh you can load; others you can only receive. Knowing which is which is part of reading this field.

FIG. 1.28 Thirteen questions across the whole chapter. They ask you to place systems I never named, using the ideas the figures were built to teach. One of them is a trap, and it is the kind of mistake I'd rather you made here than in a meeting.

Sources

A Functional Taxonomy of World Models WORLD LABS, 2026 ↗

First-party renderer/simulator/planner split, derived from the agent loop.

https://www.worldlabs.ai/blog/taxonomy-of-world-models

A New Approach to Linear Filtering and Prediction Problems KALMAN, 1960 ↗

The hidden-state ancestor. Paywalled.

https://doi.org/10.1115/1.3662552

Recurrent world models for planning and curiosity SCHMIDHUBER, 1990 ↗

A recurrent model predicting the consequences of a controller's actions.

https://people.idsia.ch/~juergen/world-models-planning-curiosity-fki-1990.html

World Models HA & SCHMIDHUBER, 2018 ↗

The paper that popularised the modern label.

https://arxiv.org/abs/1803.10122

Learning Latent Dynamics for Planning from Pixels (PlaNet) HAFNER ET AL., 2018 ↗

Raw pixels to stochastic latent state to online planning.

https://arxiv.org/abs/1811.04551

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

Behaviour learned from multi-step latent imagination.

https://arxiv.org/abs/1912.01603

I-JEPA ASSRAN ET AL., 2023 ↗

Predicting representations of masked regions, not pixels.

https://arxiv.org/abs/2301.08243

V-JEPA 2 META AI, 2025 ↗

Action-free pre-training, then action-conditioned control.

https://ai.meta.com/blog/v-jepa-2-world-model-benchmarks/

Genie: Generative Interactive Environments BRUCE ET AL., 2024 ↗

Action-controllable generated environments.

https://arxiv.org/abs/2402.15391

Genie 3 GOOGLE DEEPMIND, 2025 ↗

Reports recalling previously seen detail over multi-minute interaction.

https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

Marble WORLD LABS, 2025 ↗

Gaussian splats plus collider meshes: an explicit structural export.

https://www.worldlabs.ai/blog/marble-world-model

Cosmos NVIDIA ↗

A boundary case: predictive video worlds beside explicit simulation.

https://www.nvidia.com/en-us/ai/cosmos/

Emergent World Representations LI ET AL., 2022 ↗

Board state found and causally manipulated inside Othello-GPT.

https://arxiv.org/abs/2210.13382

Othello-GPT has a linear emergent world representation NANDA ET AL., 2023 ↗

The follow-up that sharpened the finding.

https://arxiv.org/abs/2309.00941

FIG. 1.29 I've ordered these for someone building on this rather than for historical completeness, and preferred first-party material throughout.